

Reviewer Pack — Ramanujan’s q-Series in AC⁰ Parity Lower Bounds

Entry 81d86e5e4695 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 16:07:12 UTC

Ramanujan’s q-Series in AC⁰ Parity Lower Bounds

Entry ID: 81d86e5e4695 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 16:07:12 UTC

1. Conjecture

Field A (mathematical branch): q-Series Analysis **Field B** (complexity object): AC⁰ Parity Circuit Complexity

Statement:

[‘For any constant c , there exists a polynomial-time computable quantity associated with the coefficients of an AC⁰ circuit C computing PARITY on n inputs, such that for all circuits C , if $|C| \leq n^c$, then the q-Series associated with C satisfies the inequality: $\sum_{n=0}^{\infty} (q^n)^{|C|/n} = \Theta(|C|)$.’, ‘This implies that q-Series Analysis can be used to bound the size of AC⁰ circuits computing PARITY.’, ‘If a circuit C computes PARITY on n inputs with size $|C| \leq n^c$, then the q-Series associated with C should not converge.’]

Rationale (proposer’s reasoning):

[“Ramanujan’s q-Series have been used in other mathematical fields to analyze complex structures. Exploring their application to AC⁰ circuit complexity may reveal new structural properties of circuits.”, ‘AC⁰ Parity is a fundamental problem in complexity theory, and understanding its computational hardness is crucial. The proposed invariant offers a potential avenue for improving lower bound techniques.’, ‘The use of q-Series as an invariant would be novel in the context of AC⁰ circuit complexity, potentially leading to new insights and approaches.’]

Taxonomy category: AC⁰_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d60c489e31932d91

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The q-Series associated with an AC⁰ circuit computing PARITY, if its size is $|C| \leq n^c$, will be considered supported if it converges within a predefined threshold for at least 95% of the seeds tested. It will be considered falsified if any seed produces a q-Series that does not converge or exceeds the convergence threshold.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_ac0_circuit(n):
    circuit = [random.choice([0, 1]) for _ in range(n)]
    return circuit

def q_series(circuit):
    n = len(circuit)
    series = []
    for i in range(1, n + 1):
        term = (math.exp(-i) * sum(circuit[j] for j in range(i, n, i))) / math.factorial(i)
        if term == 0:
            break
        series.append(term)
    return series

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.choice([5, 10, 15, 20, 30, 40])
    circuit = generate_ac0_circuit(n)

    series = q_series(circuit)
    if not series:
        return {
            "metric_name": "q-series_convergence",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "empty_series"
        }
    }
```

```

convergence_threshold = 0.01
if abs(sum(series)) < convergence_threshold:
    return {
        "metric_name": "q-series_convergence",
        "metric_value": sum(series),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "does_not_converge"
    }

    return {
        "metric_name": "q-series_convergence",
        "metric_value": sum(series),
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        counterexample = next(result["counterexample"] for result in results if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

.508097344109172, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'q-series_convergence', 'metric_value': 9.762683480513854, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'q-series_convergence', 'metric_value': 1.1804231110393004, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'q-series_convergence', 'metric_value': 1.6166771958910873, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'q-series_convergence', 'metric_value': 1.1796038098606108, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'q-series_convergence', 'metric_value': 2.3600830704582365, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'q-series_convergence', 'metric_value': 1.6068530479223821, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'q-series_convergence', 'metric_value': 9.300902176438871, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'q-series_convergence', 'metric_value': 3.975237586748914, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'q-series_convergence', 'metric_value': 2.3599707713415854, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'q-series_convergence', 'metric_value': 7.351472662200442, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'q-series_convergence', 'metric_value': 2.3675054644491986, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}

```

TRIAL: {'metric_name': 'q-series_convergence', 'metric_value': 2.428461156849938, 'instances_tested': 15}
RESULT: SUPPORTED mean=3.683494152416462 std=0 support_fraction=1.0

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test has only been conducted on a very small number of instances ($n \leq 15$). This is insufficient to confirm the conjecture, as it may not scale with n and could be coincidental. Additionally, the metric 'q-series_convergence' might be trivially bounded by construction, making the observed results vacuous.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test has only been conducted on a very small number of instances ($n \leq 15$), which is insufficient to confirm the conjecture. The critic's challenge regarding the scalability and potential triviality of the metric 'q-series_convergence' raises concerns about the reliability of the results. | next: Conduct further tests with a larger range of instance sizes and seeds to validate the conjecture. Investigate the nature of the 'q-series_convergence' metric to ensure it is not trivially bounded.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15077
2	preregistration	ollama_remote	glm4:latest	0	0	9560
3	novelty	ollama_remote	glm4:latest	0	0	8605
4	novelty	ollama_remote	glm4:latest	0	0	8550
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10638
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9001
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8434
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7569
9	critic	ollama_remote	glm4:latest	0	0	15506
10	judge	ollama_remote	glm4:latest	0	0	9503

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 102444 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/81d86e5e4695.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/81d86e5e4695.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/81d86e5e4695.tar.gz` (if generated)